Cardoso App — Codebase Review

Full-codebase bug / security / performance / architecture review

10 June 2026

Findings from a five-area review (backend routes, services/sync, frontend, security, architecture) across ~94K LOC. Each item has a stable reference (e.g. CRIT-1) for use in commits/PRs. Severity: P0 critical, P1 high, P2 medium, P3 low. Status starts Open; tick the checkbox in the .md as items are actioned.

41 items: 3 P0 critical 10 P1 high 18 P2 medium 10 P3 low

Critical Bugs (3)

CRIT-1 [P0] Every MSSQL pool is secretly the same global pool

Where: syncEngine.js:183, batReconciliation.js:160, jtiPool.js:88, customerSqlPool.js:79, connections.js:57

What: All six call sql.connect(config) — mssql's GLOBAL API, which ignores config if a pool already exists. Whoever connects first wins; a site with a separate BAT-only connection can import customers from the WRONG Sage DB; pool.close()/resetSagePool() in one module aborts in-flight queries in another. Likely source of 'Connection is closed' sync failures.

Fix: Use new sql.ConnectionPool(config).connect() per module; never sql.connect().

CRIT-2 [P0] set-initial-password never clears must_change_password

Where: auth.js:205; statements.js:11

What: updateUserPassword is only UPDATE "user" SET password_hash=?. The audit log claims the flag is cleared but nothing clears it — affected users are forced through the set-password screen on EVERY login, silently rotating their password each time.

Fix: Add must_change_password=0 to the password-set update (and verify hub-pushed/seeded users).

CRIT-3 [P0] Unmatched /api GETs hang forever in production (no JSON 404)

Where: server.js:292

What: The SPA catch-all does `if (!req.path.startsWith('/api')) res.sendFile()` — for /api/* it neither responds nor calls next(), and the referenced JSON 404 handler does not exist. Stale frontends hitting renamed endpoints tie up sockets until client timeout. Related: hub-redirect login GETs a POST-only endpoint (auth.js:100) so that flow cannot complete and logs the user out of the site.

Fix: Add a JSON 404 for unmatched /api/* after the catch-all; add a GET handler (or fix the redirect) for hub-token-login.

Permission Gaps (3)

PERM-1 [P1] Report JSON endpoints skip the guard their exports enforce

Where: reporting.js:1617 (aged-debtors), :1914 (rep-exposure), :1948/:2036/:2093 (BAT)

What: These are requireAuth only while their /export twins require can_access_reports. Any logged-in account can read the full debt ledger / rep exposure / BAT data as JSON.

Fix: Apply reportsGuard (can_access_reports) to the JSON endpoints to match the exports.

PERM-2 [P1] Auto-flag mutation/test endpoints are requireAuth-only

Where: records.js:634 (test-rule), :746 (apply-auto-flags), :791 (clear-auto-flags)

What: Any user can wipe every record's flags; test-rule also returns real customer balances/invoices. The hub twin correctly requires admin.

Fix: Add requireAdmin (or can_manage_rules) to match the hub equivalents.

PERM-3 [P2] Hub commission archive list/download vs bundle use different permissions

Where: hub.js:2315/2342 (can_access_monthly_reports) vs :2382 (can_access_commission)

What: Same data, two different permission keys — one is wrong. Site-side equivalents both use can_access_monthly_reports.

Fix: Standardise all three on can_access_monthly_reports.

Sync Correctness (8)

SYNC-1 [P1] Incremental sync misses updates made during the sync window

Where: hubEtl.js:181; reporting.js:2429

What: `since` is the hub-clock timestamp from AFTER the whole 30-60s sync finished, but sites filter on site-clock updated_date. Anything updated mid-sync (or under clock skew) is never re-pulled until the next Sage import rewrites it. A plausible source of the stale-data complaints.

Fix: Stamp `since` from records-fetch START minus a safety margin (or track max(updated_date) received). Upserts make re-pulls harmless.

SYNC-2 [P1] Site imports rewrite every row every 30 min, defeating incremental pull

Where: syncEngine.js:301-489

What: No change detection — unchanged rows still get a full UPDATE bumping updated_date/synced_at inside one synchronous transaction. Side effect: the hub then re-downloads the ENTIRE record set on every tick after each import.

Fix: Hash-compare `data` and skip no-op updates. Cuts site write churn and hub traffic ~95%.

SYNC-3 [P1] Hub ETL stages fail silently while the run is logged status=ok

Where: hubEtl.js (the 9 isolated pull stanzas) + 1015

What: Isolated stanzas log-and-skip to console only; a permanently-404ing endpoint (old site version) means stale hub tiles forever with zero operator signal. Violates the no-silent-failures rule.

Fix: logError per stanza failure + persist a partial-error status in hub_sync_log.error.

SYNC-4 [P2] hub_inventory prune uses one SQL placeholder per item (>32,766 SKUs fails)

Where: hubEtl.js:367-371

What: DELETE ... NOT IN (one ? per item). A site above SQLite's variable cap fails its whole sync every cycle; `since` never advances. The site side already fixed this with a temp-table anti-join (syncEngine.js:550); the hub side wasn't updated. Also: hub_records is never pruned, so cleared/renumbered records linger as ghosts.

Fix: Use a temp-table anti-join for the prune; add hub_records pruning.

SYNC-5 [P2] Hub Aged Debtors ledgerReady gate is global, not per-site

Where: reporting.js:1441-1458

What: Once ANY site's AR open-items land, sites whose ETL hasn't landed vanish from the report instead of falling back to the snapshot. Relevant to PR #389's staged rollout — it will transiently hide branches.

Fix: Make the ledger-vs-snapshot decision per site_id, not a global LIMIT 1.

SYNC-6 [P2] auto-sync-cycle fires all due imports concurrently, unawaited, errors swallowed

Where: scheduler.js:762-770

What: Two different connections run concurrently (triggers CRIT-1's pool clash); job_runs records 'succeeded' before imports finish; errors land only in console.error.

Fix: Await sequentially (like runScheduledSyncCycle) or Promise.allSettled + logError.

SYNC-7 [P2] probeSageHealth closes the shared pool on every probe failure

Where: batReconciliation.js:253

What: One 60s probe blip calls resetSagePool() !' pool.close(), aborting whatever global pool).

Fix: Mark the pool stale and let the next getSagePool() recreate; don't close under in-flight work.

SYNC-8 [P3] Cancel of Cardoso invoice generation reads the wrong (global) state

Where: batReconciliation.js:3958-4128

What: Cancellation checks read the global _activeGenerate (now nulled/replaced), so a cancelled run can complete and then wipe a newer run's lock, allowing two generations to interleave writes.

Fix: Capture a local run object and check me.cancelled; only clear the global if it still === me.

Money / UI Bugs (6)

UI-1 [P1] SA comma-decimal amount input parsed 100x too large

Where: CustomerSearch.jsx:40/57/90; lib/format.js:8

What: The amount-search input accepts commas but the parser strips them, so 11710,66 searches for R 1 171 066. parseAmount can't round-trip its own formatAmount output either.

Fix: Normalise SA format (strip spaces, comma→dot) before parseFloat; fix format.

UI-2 [P2] Snapshot aging off by one day on UTC+2 host

Where: reporting.js:94-123

What: UTC-midnight parse vs local-midnight 'today' — same-day invoices compute
bucket boundary shifts a day late.

Fix: Parse invoice dates at local midnight (or compute both in UTC) so day-diff is consistent.

UI-3 [P2] Destructive/admin actions silently no-op on failure

Where: HubDashboard.jsx:896 (force-resync), Layout.jsx:589 (app-update), Reconciliation.jsx:782 (refresh-sage), :383 (BAT settings load)

What: These do fetch with no res.ok check + bare catch. The BAT-settings one silently falls back to hard-coded TG/VAT rates and posts them into ledger generation (money-affecting). Violates no-silent-failures.

Fix: Check res.ok, surface a toast/error on failure, and fail loudly when saved rates can't load.

UI-4 [P2] keepPreviousData is dead API under react-query v5 + undebounced search

Where: CreditorSummary.jsx:192/310, CreditorSearch.jsx:48

What: v5 removed keepPreviousData (now placeholderData: keepPreviousData), so the vendor table + AP tiles blank/flash to R 0.00 on every keystroke, one request per character.

Fix: placeholderData: (prev)=>prev + a 250ms debounced value in the queryKey (CustomerBalances already does this).

UI-5 [P3] Hub renders genuine R 0.00 as "—" (hub-mirrors-site violation)

Where: HubDashboard.jsx:38-43

What: formatAmount returns '—' for 0, so a real zero balance is indistinguishable from no-data; the site UI distinguishes them, so hub and site disagree for the same customer.

Fix: Distinguish empty (—) from zero (R 0.00), matching CustomerLookup.

UI-6 [P3] Flag-count tiles show 0 when the KPI endpoints fail

Where: CustomerSearch.jsx:555-570; HubTrends.jsx:21

What: queryFns catch→return null and tiles render ?? 0, so a DB-locked error disp
affirmative wrong statement instead of an error state.

Fix: Surface an error/skeleton state instead of coercing failures to 0/[]

Security (5)

SEC-1 [P1] Sage override validator is bypassable

Where: queryRegistry.js:476; commission.js:40

What: Keyword denylist that requires a leading SELECT/WITH does NOT block SELECT…INTO, OPENROWSET/OPENQUERY/BULK, WAITFOR DELAY, or (it allows leading ;) multi-statement. Admin-only, but the guardrail's job is to keep overrides read-only even for admins; if the Sage login has write rights this mutates the ERP DB.

Fix: Run overrides through a least-privilege READ-ONLY Sage login (DB enforces it); strip comments + block INTO/OPENROWSET/WAITFOR/xp_/sp_/second-statement as defense in depth.

SEC-2 [P2] Auto-update trusts GitHub release contents end-to-end (no signature check)

Where: system.js:80-91, 1218, 1297, 1629

What: Installer + checksum come from the same release; the .exe is run via Task Scheduler as SYSTEM, auto-hourly. A compromised repo/account propagates fleet-wide within an hour. (Repo pinned + host allowlist + HTTPS are good controls already.)

Fix: Verify Authenticode signature + publisher thumbprint before launch; sign the manifest with an off-repo key; prefer operator-approved tag over auto-latest.

SEC-3 [P2] Token compares non-constant-time; /api/backup/config defaults to FULL .env off-production

Where: reporting.js:156, backup.js:51/118

What: Token gates the full-DB download and the unredacted .env (SESSION_SECRET, ENCRYPTION_KEY). !== is timing-leaky (low risk over Tailscale). getBackupConfigExportMode() returns 'full' whenever NODE_ENV !== 'production' — a footgun if the service starts without that env.

Fix: crypto.timingSafeEqual (length-guard first); default config export to redacted/disabled, explicit opt-in for full.

SEC-4 [P3] Production CORS reflects any Origin with credentials

Where: server.js:119-125

What: origin callback always allows; currently blunted by sameSite:'strict' so it's latent, but any future cookie relaxation makes it cross-origin readable.

Fix: Reflect an explicit allowlist (own origin + hub origin) instead of cb(null,true).

SEC-5 [P3] Session cookie secure=off on default LAN-HTTP install; raw errors returned to clients

Where: server.js:189; records.js:1363, reporting.js:951/992, server.js:284

What: Default HTTP install sends the session cookie cleartext on the local LAN segment. Several handlers return raw err.message (SQL/table fragments).

Fix: secure:true once TLS lands; return a generic message + keep detail in logError.

Performance (4)

PERF-1 [P1] Make precomputed rollup tables THE pattern for heavy reads

Where: reporting.js top-balances/top-items-mtd/dead-stock/aged-debtors; hub.js (28 GROUP BY endpoints)

What: Synchronous better-sqlite3 aggregation on the request path is the class that caused this week's production hangs. v099 point-fixed two endpoints; ~30 more aggregate per request (some load every row + JSON.parse, some run window functions over the whole item master).

Fix: Generalise the v099 rollup approach: rebuild at sync-end, both UIs read the same table; add rollup_meta staleness to /api/system/health.

PERF-2 [P2] Hub full-refreshes 7 datasets every 5 min when data changes nightly

Where: hubEtl.js:417-767

What: movement/item-sales/customer-sales/AR/AP/creditor/stock-receipt staged fully in memory + delete-all+reinsert, per site, ~288x/day, blocking the hub event loop — but the source changes once nightly.

Fix: Move these to hourly/nightly cadence or gate on a site-reported freshness stamp; keep only records/KPIs/BAT on the 5-min loop.

PERF-3 [P2] Heavy per-search / per-request scans

Where: records.js:332 (customer-by-amount, 20K rows x2 JSON.parse), batReconciliation.js:557 (integrity report N+1), inventoryMovement.js:13 (3yr window staged in one txn)

What: Operator-triggered or polled endpoints doing full scans / per-row JSON.parse / N+1 invariant checks synchronously on the main thread.

Fix: Trigger-maintained side tables for amount search; cache integrity results keyed on recon last-modified; trailing 1-2 month window for nightly inventory after backfill.

PERF-4 [P3] Frontend: undebounced search, idle 2s polling, unvirtualized vendor table

Where: CreditorSummary.jsx:310/439, OcrPanel.jsx:102

What: One request per keystroke; OCR snapshot polls every 2s while idle (~45 req/min from one tab); CreditorSummary renders the full vendor table without @tanstack/react-virtual (which other tables use).

Fix: Debounce keys; make refetchInterval a function of running state; virtualize the table.

Architecture (7)

ARCH-1 [P1] Define the site!'hub dataset contract once

Where: hubEtl.js syncSite (9 copy-pasted pull stanzas) + reporting.js (8 copy-pasted serve endpoints)

What: One logical thing written ~20 times; field drift is invisible until a hub tile goes blank (the last three incidents). New dataset costs ~300 copy-pasted lines.

Fix: A HUB_DATASETS config table + one pullDataset() engine + a generated site router. Add a CONTRACT TEST per dataset (site fixture!' pull!' assert hub table) — the single highest-value missing

ARCH-2 [P1] Commit to rollup tables as the pattern (retire per-request aggregation)

Where: see PERF-1

What: Structural fix for the event-loop hang class; both hub+site reading one rollup enforces hub-mirrors-site by construction.

Fix: reporting_rollups.js with rebuildX() called at sync-end; migrate dashboard + aged-debtors/creditors + hub trends in small PRs.

ARCH-3 [P2] One isHubMode() instead of 40 scattered env checks

Where: ~40 process.env.HUB_MODE==='true'; 4 private isHub() copies (debtors/creditors/stockReceipts/inventoryMovement)

What: config/env.js validates HUB_MODE then everyone re-reads process.env raw. 18 reporting endpoints carry full dual implementations inline.

Fix: Export isHubMode() + split report builders' data acquisition per mode behind a small source interface; keep compute single-pathed.

ARCH-4 [P2] Split the giants along their natural seams; extract builders to services

Where: reporting.js (2995), hub.js (3722)

What: reporting.js is 3 modules (user reports / pure builders / the token-auth machine contract); builders being inline is why none are tested.

Fix: Move-only PRs: builders!'service (+ first unit test), /api/reporting/*!'own route

ARCH-5 [P2] Highest-value test gap: hubEtl + dataset contract, then builders

Where: hubEtl.js (0 tests), report builders (0 tests), React (1 test / 186 files)

What: Coverage is inverted relative to risk — every incident this week was in untested code.

Fix: Contract test (ARCH-1), syncSite orchestration tests with stubbed fetch, builder unit tests as extracted.

ARCH-6 [P3] Write the migration index generator the header already claims exists

Where: db/migrations/index.js (refs missing scripts/_extract-migrations.mjs)

What: 99 hand-maintained imports; a file present on disk but absent from index silently never runs (the failure shape the v62 gate was written for).

Fix: Write _extract-migrations.mjs (readdir+emit); extend the CI gate to diff and fail on a missing index entry.

ARCH-7 [P3] Standardise the route error funnel on logError + verbose messages

Where: reporting.js (12 logError vs ~24 raw console.error); some return raw err.message

What: Error handling is bimodal — hubEtl is the house-rule exemplar; many route catches discard the message and log nothing to the System Log.

Fix: routeErrorHandler(scope) helper that always logErrors with context + returns a stable message + error id; sweep one file per PR.

Feature Recommendations (5)

FEAT-1 [P2] Ops alerting on sync failures / staleness

Where: new

What: Sync failures land in the System Log only; nobody reads logs until a tile looks wrong.

Fix: Daily digest + immediate email/webhook on 'site unreachable >30 min' or 'dataset stale >24h'. Would have caught this week's issues early.

FEAT-2 [P2] Fleet version / freshness dashboard

Where: new (hub)

What: The hub knows each site's version + last-seen + data freshness.

Fix: One Operations panel: version / last-seen / freshness / pending update per site — replaces the 'did 5.9.1 land everywhere?' guesswork.

FEAT-3 [P2] Automated hub!”site reconciliation check

Where: new (nightly job)

What: Mechanically enforce the hub-mirrors-site rule instead of relying on review discipline.

Fix: Nightly compare per-site totals (debtor balance, AP outstanding, inventory value) hub-vs-site; flag drift.

FEAT-4 [P3] Staged / canary auto-update rollout

Where: system.js auto-update

What: A bad build currently fans out fleet-wide within the hour.

Fix: Update one canary site, verify health + version report, then fan out. Would have contained the 5.9 hang to one branch.

FEAT-5 [P3] Aged-debtors !’ collections worklist bridge

Where: new

What: You already have collections worklists + per-document aging.

Fix: Auto-feed 'new 21+ bucket entries' into a collections worklist — a small join with real business value.